Colegio Connect — Capstone Progress Report

Fall 2026 · Report date: November 10, 2026

Gabriel E. E. Rodríguez Rodríguez · Cristian J. Massini Pérez · Luis A. Marrero Morales

Department of Computer Science and Engineering, University of Puerto Rico at Mayagüez

Abstract

Colegio Connect is a web prototype that brings six kinds of academic and professional opportunity (jobs, internships, COOP positions, research openings, research participation, and capstone projects) into one discovery experience for UPRM students. The design combines institutional email verification, organization approval, server-side access control, consent-limited student disclosure, moderation with audit history, and two routes for capstone sponsorship. The approved proposal defines five objectives measured by 17 indicators [1].

At this checkpoint, the main branch (commit f0c4efe) contains the access foundation and the first discovery slice. The Java/Spring Boot/PostgreSQL backend provides registration with email verification, server sessions with CSRF protection and lockout, role-based authorization, moderator role management, a transactional audit log, the company approval workflow, and a six-category discovery feed with keyword, category, publisher, and deadline filters that enforces publisher trust rules. The React/TypeScript frontend provides sign-in and registration screens and a dashboard for each of the four roles. Hosted CI on this revision passed all four jobs with 112 backend and 177 frontend tests. Local measurements recorded 210 successful authentication requests and three successful PostgreSQL restart trials.

Listing creation, saving, student responses, reporting, and both sponsorship routes are not yet implemented, so the project remains behind its planned milestones. This report addresses the proposal review, updates the design, reports preliminary results with screenshots of the running system, and presents a recovery schedule through the December 12 final deliverables.

1. Introduction

UPRM students find jobs, internships, COOP positions, research openings, research participation, and capstone projects through separate offices, departments, and external sites. The UPRM Placement Office and Open Capstone are useful examples, but each covers only part of this space [1], [2], [3]. A student must know which channel to watch and when. The problem is also one of trust: an offer shared outside an official channel can be hard to verify. Research links internship experience and undergraduate research to better outcomes, and the FTC has warned of employment scams aimed specifically at college students [4], [5], [6].

Colegio Connect is a campus-focused, multi-role prototype. Verified students discover and respond to listings. Authorized institutional publishers and approved companies create listings. Moderators approve organizations and act on reports. Student teams can publish a capstone project and decide whether to

accept a sponsor's interest. The system is an independent academic prototype, not an official UPRM service, and does not integrate with university identity systems [1].

This report describes the system on the main branch at commit f0c4efe as of November 10, 2026. Appendix A lists the pull requests and verification runs behind each result.

1.1 Response to the proposal review

The course requires this report to address the proposal review, the updated design, preliminary performance, and repository and web-page documentation [7], [8]. The proposal evaluation raised three issues [9]:

Reviewer comment	Response in this report
No UI sketches or wireframes	Figure 6 presents the student discovery wireframe; wireframes for all four roles are on the project web page (Section 8.1), and Figures 7–9 show the implemented screens.
Engineering constraints mixed with management constraints	Section 5.7 separates schedule, team size, and budget from technical requirements, and gives each technical requirement a target and a verification method (Table 2).
Incorrect citation numbers, especially in Section 5(f)	References were rebuilt from the original sources and numbered in order of first appearance.

2. Problem Statement

The target users are UPRM students, institutional faculty, offices and departments, external companies, and moderators. Students are the main beneficiaries; publishers and moderators make the feed useful and trustworthy. The approved access model has two account types, UPRM_MEMBER and EXTERNAL_ORGANIZATION, and four roles: Student, Faculty/Office/Department, External Company, and Admin/Moderator. Each person has an individual login and may hold more than one role. Organizations are separate records, and a company must be approved before it can publish [1], [10].

Existing UPRM channels each address part of the problem: PRJobs covers jobs, the UPRM Research Portal covers research connections, and Open Capstone covers engineering project sponsorship [3], [11], [12]. DeLorme's Capstone Marketplace is prior work on online sponsor matching [13]. Our contribution is not the feed or sponsor matching alone, but their combination in one six-category discovery, approval, privacy, and moderation model. The single-campus focus is also a limitation: coverage depends on publishers joining, and approving an organization does not verify the substance of every listing it posts.

Privacy is part of the problem definition:

- Companies cannot browse the student population.
- A student response discloses only the fields the student chooses.
- A student team's private contact fields are released to an interested company only after the team accepts.

Out of scope are AI recommendations, in-app messaging, interview scheduling, automated scam detection, appeals, and direct integration with UPRM information systems [1]. Human-subject research recruitment and existing application processes stay with their institutional owners; the platform links out to them.

3. Project Objectives and Performance Indicators

The approved proposal sets five objectives over four two-week sprints, measured by indicators I1–I17 [1], [10]. Table 1 keeps their IDs and deadlines and adds the current status. The full indicator targets and test procedures are in the traceability and test registers [10], [14].

Table 1. Approved objectives and status at November 10, 2026.

ID	Objective (deadline)	Indicators	Status	Evidence so far
O1	Unified discovery of six categories: search, filter, open, save; two-second response target (Week 4)	I1–I4	In progress	Listing schema and six-category discovery feed with keyword, category, publisher, and deadline filters are on main. Listing creation, details, and saving are not built.
O2	Email verification and server-side role, ownership, approval, account-status, and consent checks; zero unauthorized student disclosure (Week 6)	I5–I7	In progress	Registration, verification, sessions, role authorization, and role management are on main. Listing ownership, response consent, and full privacy acceptance remain open.
O3	Reporting, moderation, restrictions, and complete audit trail without a UPRM office (Week 6)	I8–I9	In progress	Transactional audit log, company approval decisions, and the moderator review queue are on main. Reporting and listing moderation are not implemented.
O4	Company- and student-originated capstone sponsorship with controlled contact release (Week 8)	I10–I12	Early	Verification email and notification delivery are on main. No sponsorship flow yet.
O5	Deployed acceptance, security, accessibility, recovery, and usability results (Week 8)	I13–I17	Early	CI runs on every pull request and merge; local recovery test done; deployed testing not started.

Two indicators deserve emphasis. I3 requires at least 95% of discovery requests within two seconds and under 1% server errors with 5,000 synthetic listings, 1,000 users, 50 organizations, and 25 concurrent virtual users [1], [14]. The discovery feed now exists, but this load test is scheduled after listing creation so that the dataset is produced through the real publishing path. I6 requires at least ten negative-authorization cases across protected business actions, including publishing, listing ownership, response consent, and sponsorship acceptance. Current denial tests cover identity, role management, company review, and dashboard access; the remaining business-action cases follow once those features exist.

4. Solution Approach and Previous Work

The platform has five linked parts: access and trust, structured publication, discovery and response, two-sided capstone sponsorship, and moderation with audit history [1]. A six-category feed lets students compare opportunities without learning each publisher's channel. Source labels and organization approval show who is offering a listing. Student-controlled response fields and a team acceptance gate limit what companies see. Moderation and audit history support corrective action when a listing is reported. Whether this improves the student experience will be tested in the usability study (I17).

4.1 Technology choices

Layer	Technology	Why
Backend	Java 25, Spring Boot 4.1.2, Spring Security, JPA, Flyway, Actuator, Mail, springdoc 3.2.0	Mature transactional framework; team experience; built-in session and CSRF security
Database	PostgreSQL 18.6	Relational constraints fit accounts, organizations, roles, and listings
Frontend	React 19, TypeScript 6, Vite 8, React Router, openapi-fetch, Vitest	Reusable role-specific views with compile-time type checking

Layer	Technology	Why
Infrastructure	Docker Compose (backend, PostgreSQL, Mailpit), GitHub Actions	Reproducible local integration and automated backend, frontend, and container verification

Versions are those in the repository manifests at f0c4efe [15].

4.2 Trade-offs

- **Generated typed client.** Keeps frontend calls aligned with the OpenAPI contract, but every API change requires regenerating the schema and a drift check.
- **Server sessions with CSRF protection.** Keep authentication secrets out of browser storage, but cookie and cross-origin behavior must be verified in the deployed environment.
- **Flyway migrations.** Give an ordered schema history, but rolling back application code does not roll back a committed migration.
- **Provider-independent mail interface.** Limits coupling to one email sender, but cannot guarantee delivery by the external provider.
- **Single service instead of microservices.** Manageable for three developers and lets an approval decision and its audit record share one transaction.

4.3 Operating requirements, licensing, and related work

Running the system requires a host for the browser app and API, persistent PostgreSQL storage with backups, an external email sender, TLS and secrets management, monitoring, and staff time for publisher verification, moderation, and support. Compose and Mailpit cover local development only. Source code, diagrams, and wireframes are team work; frameworks carry their own licenses, and listing text and logos belong to their submitters. Before any public release the team must review dependency licenses, rights to submitted content, and permission to use UPRM marks. No university endorsement is claimed [1].

The Placement Office, PRJobs, Research Portal, and Open Capstone each show a narrower channel or workflow [2], [3], [11], [12]. A campus-wide combination is useful only if publishers participate and students can reliably find relevant listings; the discovery test cases, load tests, and usability sessions in Section 6.5 will test that.

5. Updated Technical Approach

5.1 Architecture

Figure 1 updates the proposal's client-server diagram to show the modules that now exist. The main branch contains the React single-page app, the Spring Boot API, the identity database, the generated OpenAPI document, an SMTP mail adapter (Mailpit in development), and the CI jobs.

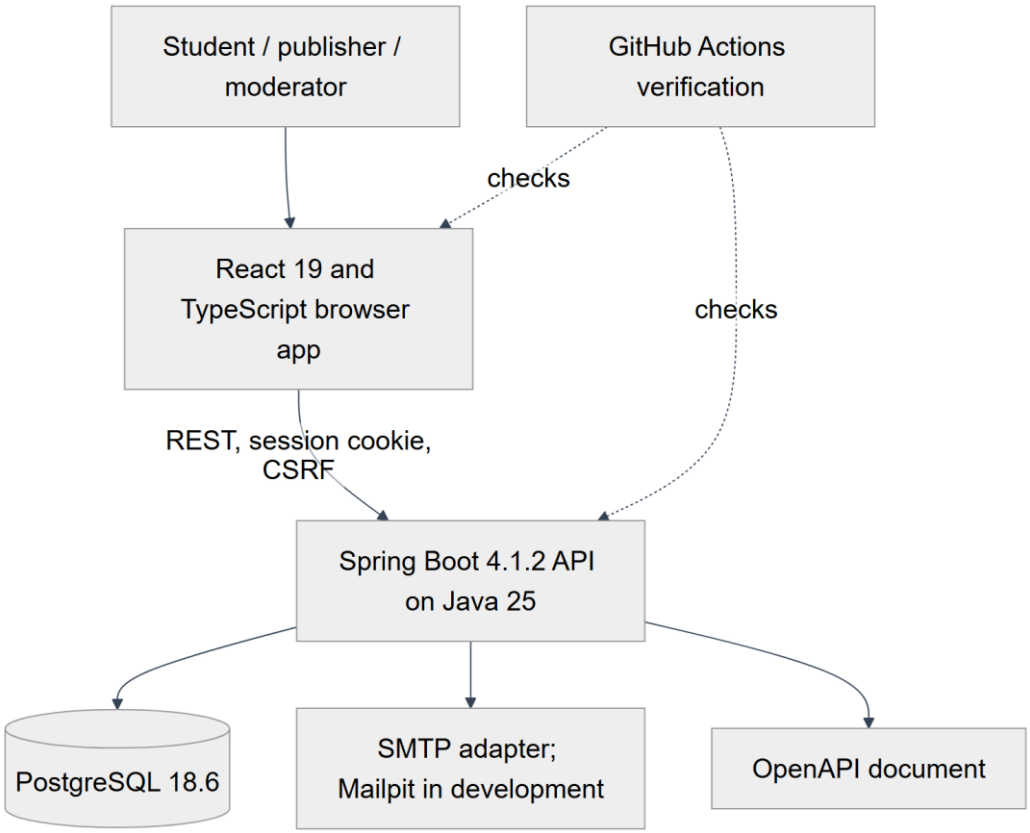


Figure 1. Updated high-level architecture. Solid arrows show runtime integration on main; dotted arrows show CI checks, which verify builds and tests but do not serve traffic.

Figure 2 shows module dependencies on main. On the frontend, AuthProvider restores the server session and role-aware routes control navigation; the API makes every authoritative decision. Controllers call AuthorizationPolicy on each request to check persisted account status, role, and organization membership. The administration services record role changes and company decisions through AuditService. The discovery feed applies publisher trust rules in its query. Listing ownership and consent checks will be added with the planned listing and response modules.

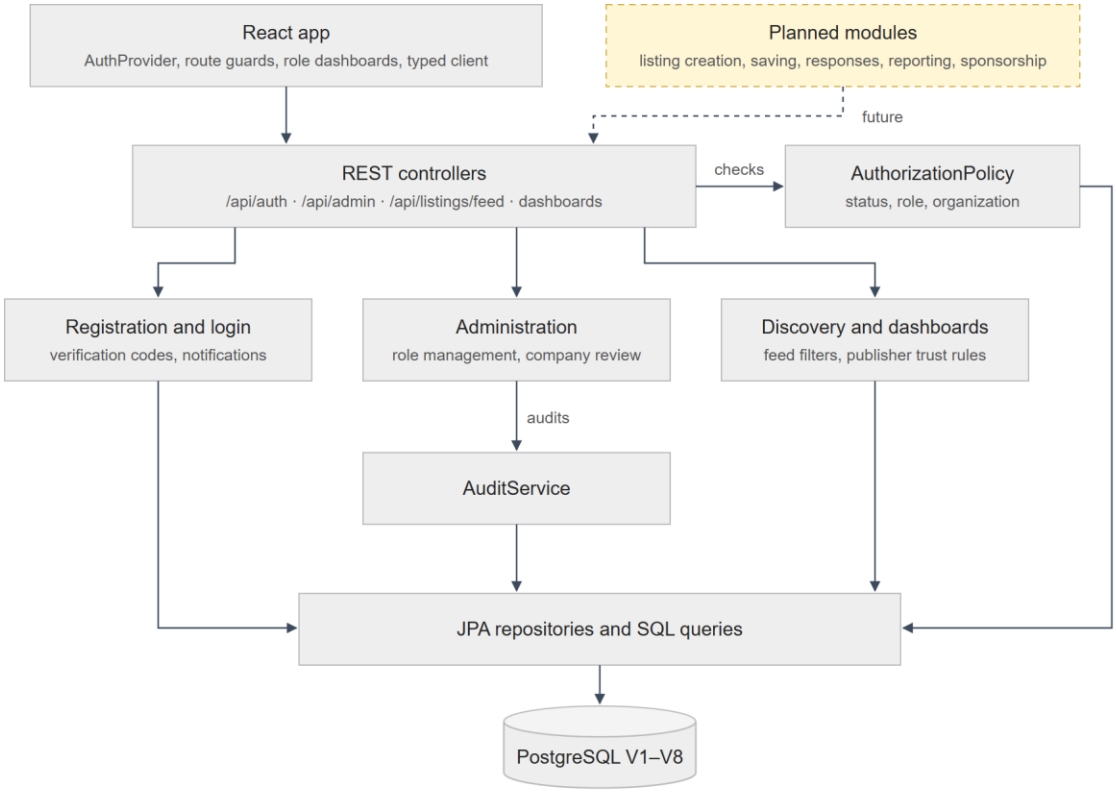


Figure 2. Module dependencies on main (f0c4efe). The dashed box marks planned modules; all other modules are merged.

5.2 Data model

Migrations V1–V3 create the identity tables (users, organizations, role_assignments, email_codes) with verification-code limits and login lockout. V4 adds audit_log and V5 adds organizations.decision_reason. V6 adds capstone_teams and listings, with the six opportunity types, a publication status, and a trust label. V7 adds saved_listings and listing_responses, whose shared_fields column holds only the fields a student chooses to share. V8 adds listing_reports, company_application_owners, and organization review time. The backend test suite applies all eight migrations to an empty PostgreSQL 18.6 database through Testcontainers and passed in hosted CI on f0c4efe. Figure 3 shows the identity core.

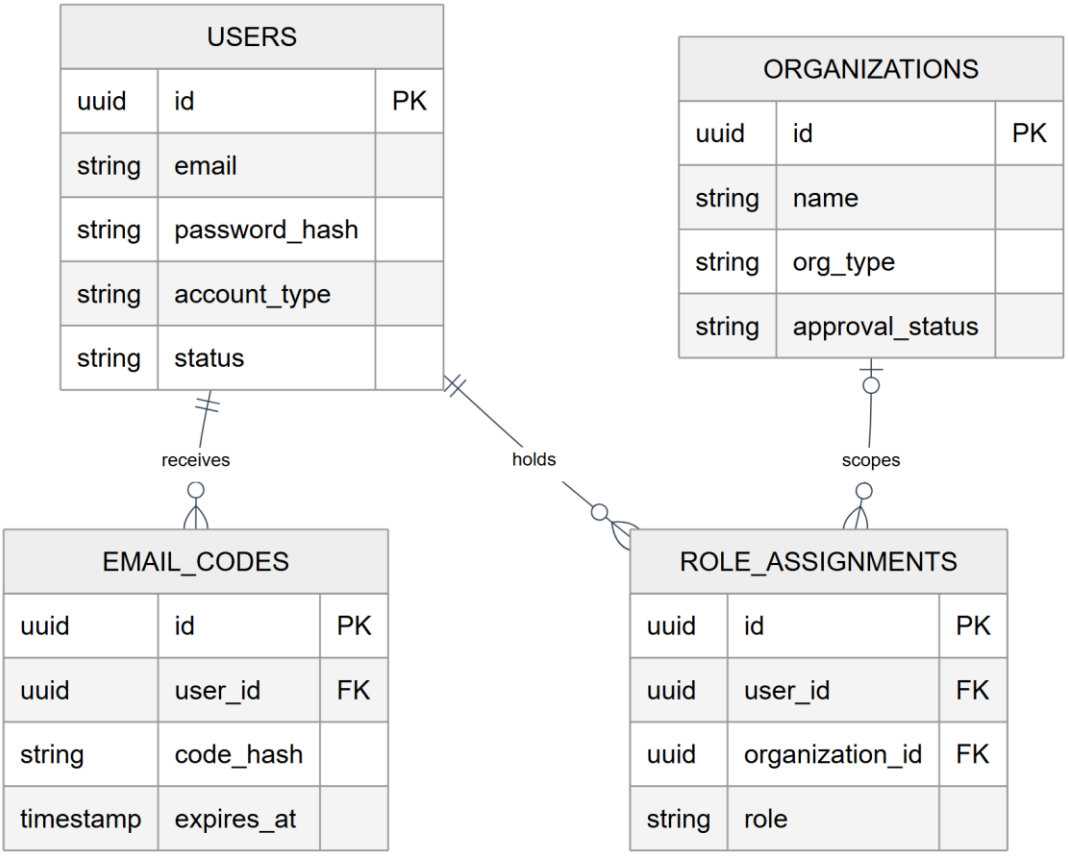


Figure 3. Identity core of the schema (V1–V3). Field lists are abbreviated; the migration SQL is authoritative. Organization-scoped role assignments support institutional and company membership; the moderator and student roles have no organization. Database role names STUDENT, INSTITUTIONAL, COMPANY, and MODERATOR map to the proposal's four roles [10], [15].

Company decisions and their audit events share a transaction, so a failed audit write rolls back the decision. Before V4 and V5 were merged, an upgrade test applied them to a database holding existing V1–V3 identity data and passed.

5.3 Authentication, authorization, and privacy

Figure 4 shows the implemented access path. Registration stores a hashed verification code with expiration and attempt limits. Login uses Spring Security with a server session, CSRF tokens on all writes, and account lockout after repeated failures. The frontend's typed API client attaches the CSRF header automatically and refreshes the token after logout. Frontend route guards improve navigation, but the API independently denies any invalid action [15].

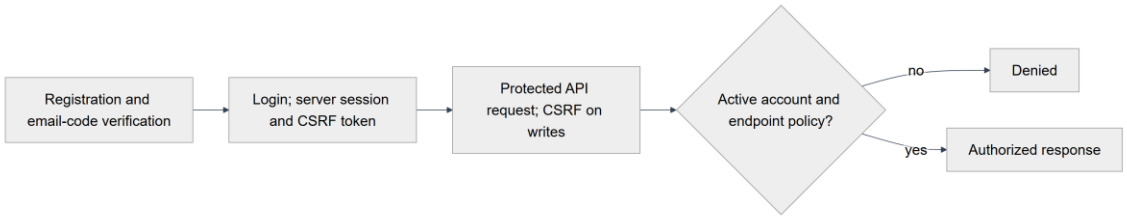


Figure 4. Authentication and authorization flow. Mail transport and error detail are omitted.

The design requires server-side checks on role, organization, approval, account status, ownership, and consent. The discovery feed already enforces publisher trust: an organization's listing appears only if its publisher holds an INSTITUTIONAL or COMPANY role in that organization and, for companies, the organization is approved; a student project appears only if its owner holds the STUDENT role. Two privacy rules remain open until their features exist: (1) listing creation must check membership and approval before publishing, and (2) no student response or team contact may be released without the consent rule. Both will receive dedicated negative tests (Section 6.5).

5.4 Company review, role management, and audit

The company review workflow (PRs #118–#120) provides a moderator-only pending-company queue and approve/reject decisions. A decision locks the organization row, allows a single transition out of PENDING, and writes one audit event in the same transaction. Tests cover denied and revoked moderators, concurrent and conflicting decisions, and rollback on audit failure. Integrating this work exposed an audit pagination defect when events shared a timestamp; a PostgreSQL regression test reproduced it, and using the event ID as a tie-breaker fixed the ordering. Role assignment and revocation (PR #124) write audit events the same way, and the first moderator is created through a one-time bootstrap setting that is itself audited.

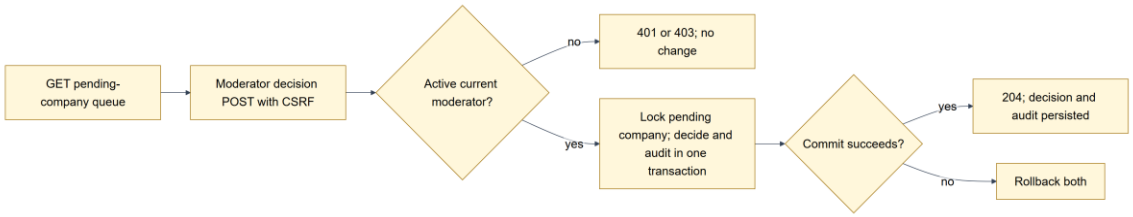


Figure 5. Company-review transaction. A failed commit rolls back both the decision and the audit entry.

Two gaps remain. Company registration does not yet record the applicant owner link that V8 provides for, so moderators cannot see who submitted a company, and the approve and reject buttons in the moderator screen (Figure 9) are not yet connected to the decision endpoint. Company approval also does not by itself grant publishing rights; listing creation must check membership and approval.

5.5 Frontend and wireframes

Low- and high-fidelity HTML wireframes cover four roles: student discovery, institutional overview, company dashboard, and moderator review queue. Figure 6 shows the student discovery view: search and filter row, source-labeled cards, Details and Save actions, and an activity panel. All content in the wireframes is placeholder.

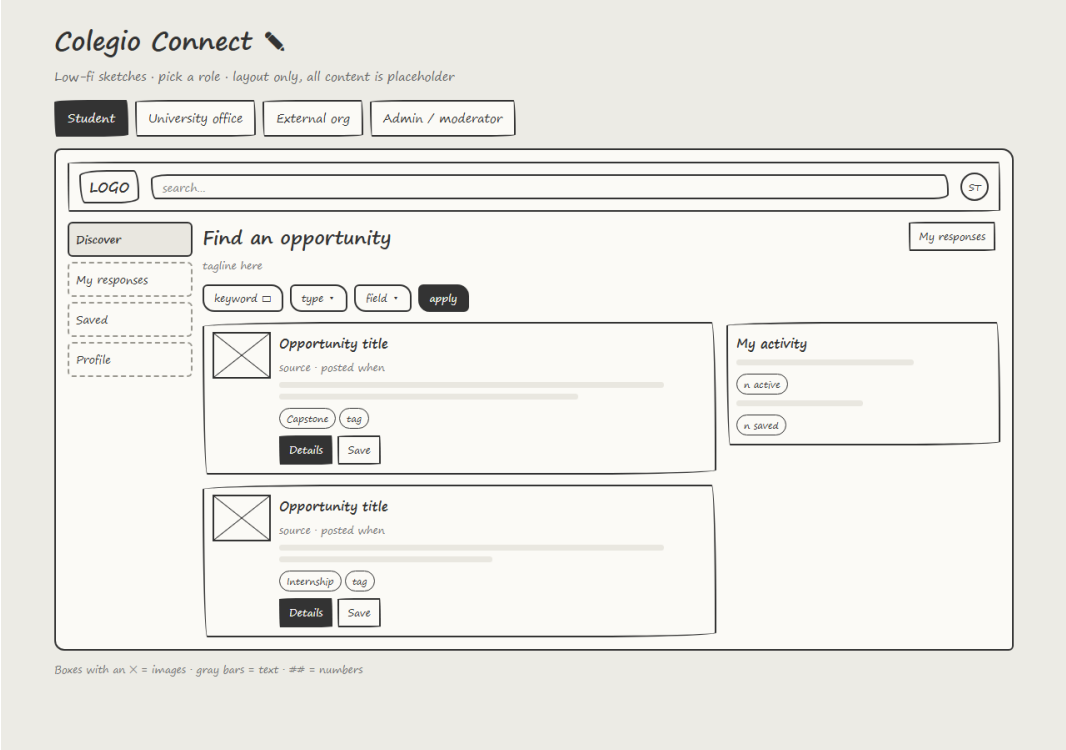


Figure 6. Low-fidelity student discovery wireframe (docs/Wireframes/lowfi-sketches.html). Interactive four-role and high-fidelity versions are in the repository.

The merged frontend provides the Vite/React app, route guards, session restore, shared layout and form components, the typed OpenAPI client, the sign-in, registration, and verification screens (PR #116), and a dashboard for each role connected to the backend read APIs (PR #128). Section 7.1 shows these screens running.

5.6 Standards and engineering practices

- **OpenAPI.** The API is described by an OpenAPI document, from which the frontend client is generated. This gives reviewers a concrete contract and lets CI detect drift. Consistent status codes, validation errors, pagination, and authorization semantics are still needed across endpoints.
- **Relational design and migrations.** PostgreSQL constraints and ordered Flyway migrations suit the account, organization, role, and listing relationships. Testcontainers runs tests against real PostgreSQL rather than an in-memory substitute.
- **WCAG 2.2 AA.** Adopted for perceivable, keyboard-operable workflows [1]. Semantic controls and component tests are early steps; automated axe scans plus manual keyboard checks will verify conformance (I15).
- **OWASP ASVS Level 1.** Used as the security checklist [1]. Session, CSRF, access-control, and negative tests address part of it; an authenticated ZAP scan and the full checklist remain (I14).
- **Testing layers.** JUnit unit tests, PostgreSQL integration tests, Vitest component tests, browser acceptance tests, and CI regression gates. The first three are in place; acceptance tests follow once the core features exist.

5.7 Engineering constraints

The proposal's C1 (eight-week schedule), C2 (three developers), and C3 (near-zero budget) are project and resource constraints: they shape sequencing and hosting choices but are not technical quality properties. Table 2 lists the technical requirements, each with a design response, a measurable target, current evidence, and the remaining verification [1], [14].

Table 2. Technical requirements, targets, and verification.

Requirement	Design response	Target and current evidence	Remaining verification
Accessibility: every role needs usable navigation, forms, and error recovery	Semantic labels, focus treatment, responsive wireframes	I15: WCAG 2.2 AA, zero unresolved Critical/Serious findings, keyboard completion. Component tests only so far.	axe scans and manual keyboard tasks on all primary screens and viewport sizes
Discovery latency under load	Pagination, indexed queries, small payloads	I3: 95% of requests < 2 s, < 1% errors, 25 virtual users. Not yet measurable.	Seed 5,000 listings / 1,000 users / 50 orgs; k6 run recording p95, error rate, CPU, RAM
Availability and fault tolerance	Persistent volume, health probe, recovery script, backup policy	I16: healthy within 5 min, no data loss. Local: login ready 1.05–1.11 s after restart; data intact in 3/3 trials [16].	Repeat on deployment with representative data and backup/restore
Student privacy and access control	Server sessions, membership checks, minimal DTOs, approval and consent gates	I6, I7, I11. Negative tests cover identity, role management, and dashboard access; the feed enforces publisher trust rules.	Ownership, response-field, team-membership, and contact-release tests
External mail reliability	Hashed short-lived codes, rate limits, provider abstraction, delivery logging	I12: all required messages sent. Verification email works.	Test every message type, retry and failure handling
Moderation without an institutional office	Moderator queue, report lifecycle, transactional audit	I8, I9. Company decisions and role changes are audited on main.	Every administrative action and report/hide/remove flow end to end
Storage capacity	Indexed schema; no attachments planned	Not yet measured	Record database and backup size with the I3 dataset

HTTPS, private database access, secrets management, and secure cookie settings will be verified on the deployed host.

6. Project Plan and Development Progress

6.1 Milestones and progress

The approved plan comprises four two-week sprints: M1 access foundation, M2 publishing and discovery, M3 responses and moderation, and M4 sponsorship and acceptance [1]. As of November 10, M1 is complete and the read side of M2 discovery is on main, but listing creation, saving, and the remaining M3 and M4 features are not. The project is roughly one sprint behind plan.

Table 3. Approved milestones and progress.

Milestone	Approved deliverable	Status	Done	Remaining
M1 — Access foundation	Running frontend/API/database; login; role checks; first negative tests; CI; wireframes	Complete	Identity schema, registration and verification, login, authorization policy, role management, auth screens, notifications, CI, wireframes	Hosting

Milestone	Approved deliverable	Status	Done	Remaining
M2 — Publishing and discovery	Company registration and approval; six-category publishing; feed, search, filters, saving; initial performance, security, accessibility results	In progress	Listing schema; six-category discovery feed with filters and trust rules; company approval backend; role dashboards	Listing creation, details, and saving; company applicant link; k6, ZAP, axe
M3 — Responses and moderation	Response privacy, reports, restrictions, audit, notifications, company capstone route, usability plan	Started	Audit log; moderator review queue; schema for responses and reports	Responses with consent; reporting and listing moderation; approve/reject screens; usability plan
M4 — Sponsorship and acceptance	Student projects, Express Interest, accept/decline with contact release; all indicators; user guide; final report	Not started	—	All

6.2 Revised schedule

Our recovery plan prioritizes completing the discovery workflow (listing creation, details, and saving) because it is required for I1–I4, privacy testing against real responses, and I3 load testing. The dates below are planning targets through the December 12 final deliverables.

Proposed dates	Work	Owner
Nov 11–15	Assign the STUDENT role at registration; add an endpoint listing a user's organizations; record the company applicant at registration; draft and submit the usability-study protocol for advisor review; decide PRs #107, #113, and #114.	All
Nov 16–22	Implement listing creation with membership and approval checks, listing details, and saving; connect the moderator approve/reject screens.	Cristian (API); Luis (UI); Gabriel (security)
Nov 23–29	Implement responses with field-level consent, report/moderation flow, and cross-role business authorization tests.	Cristian; Gabriel; Luis
Nov 30–Dec 6	Implement both capstone sponsorship routes and contact-release gates; deploy integrated application; start k6, ZAP, axe, and recovery tests.	All
Dec 7–12	Complete representative usability sessions and 20-scenario acceptance suite; address findings and prepare final report, user guide, presentation and video.	All

The Nov 30–Dec 6 week carries the most risk; if it slips, sponsorship testing moves into Dec 7–12. Beyond that, the team will first cut work outside the approved objectives (for example, interface polish beyond WCAG requirements). Any change to an approved objective, such as the six listing categories or the two sponsorship routes, would be proposed to the advisor before it is made, and all privacy rules stay in scope regardless.

6.3 Team responsibilities

The proposal assigns Cristian backend and data leadership, Gabriel infrastructure, security, and moderation, and Luis frontend and user experience [1]. Actual work has followed that division.

Member	Merged into main	Work awaiting integration	Next

Member	Merged into main	Work awaiting integration	Next
Cristian Massini	Identity migrations V1–V3, registration and verification, sessions and CSRF, authorization policy, CSRF client fix, role management with audit and first-admin bootstrap (#124), roles in login and /me (#125)	—	Listing creation, responses, student sponsorship
Luis Marrero	React/Vite app, typed client, route guards; sign-in, registration, and verification screens (#116); notification delivery (#115); dashboards for all four roles (#128); four-role wireframes; project web page	—	Listing details and saving UI, moderator approve/reject screens, company capstone route
Gabriel Rodríguez	Compose and CI; negative-authorization harness (#118); audit log and company review (#119, #120); listing schema, discovery feed, and dashboard read APIs (#127); local recovery rehearsal	#107 CI triggers; #113 frontend Compose; #114 recovery evidence	Moderation, security testing, deployment

Detailed commits, PRs, and test results for each contribution are in Appendix A.

6.4 Development process

Work proceeds through topic branches, pull requests, and GitHub Actions. CI runs backend, frontend, and container jobs and a CI Gate on every pull request to main and every merge; PR #107, still open, would extend this to every branch push and add manual runs. The team requires peer review and passing CI before merging, including regenerating the OpenAPI client when contracts change. Authorization and contact-release changes require an additional security review. The immediate interface dependencies are the student role assignment, an endpoint listing a user's organizations, the company applicant link, and the listing write contracts.

6.5 Verification plan

Each acceptance criterion is linked to a synthetic fixture, procedure, expected result, and the revision tested. The test layers are:

- **Unit (JUnit):** validation and decision rules.
- **Integration (Spring + PostgreSQL Testcontainers):** migrations, constraints, transactions, rollback.
- **Frontend (Vitest):** component state and error paths.
- **Acceptance (browser + API):** the 20 core scenarios of I13, once listings, responses, moderation, and sponsorship exist.

For the indicators specifically, the plan adds negative security cases across roles and organization states, checking both the denial and that data is unchanged; an authenticated ZAP scan; axe plus manual keyboard and responsive checks; k6 load runs with the I3 dataset and recorded host resources; timed restart and integrity checks on the deployment; and six usability sessions under an advisor-approved protocol with anonymized results [1], [14].

7. Preliminary Results

7.1 Implemented functionality and tests

Main revision f0c4efe provides registration and email verification, login and session endpoints, role-based authorization and role management, the audit log, company review, the six-category discovery feed, and dashboards for all four roles [15]. Hosted CI on this revision passed all four jobs (Table 4). Figures 7–9 show the running application, captured on November 10 from f0c4efe in an isolated local Docker Compose project. Accounts, organizations, and listings in them are synthetic data inserted directly into that database, because listing creation is not built yet.

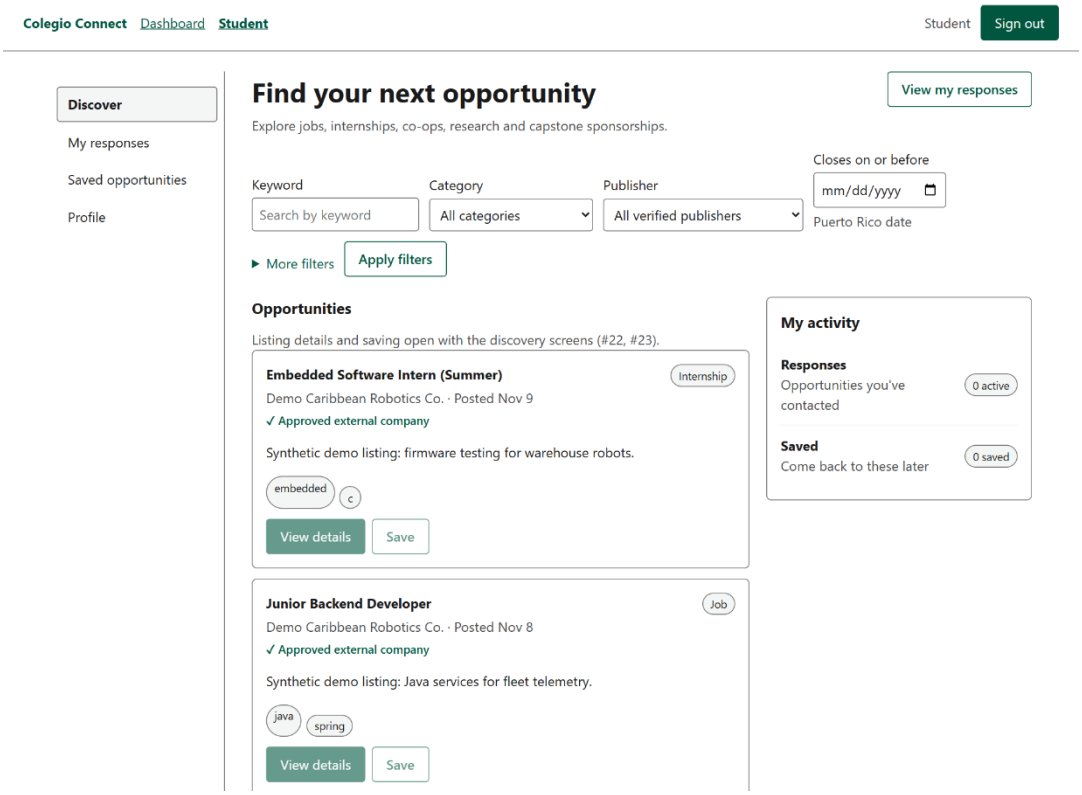


Figure 7. Student discovery feed on main (f0c4efe). Filters for keyword, category, publisher, and closing date; each listing shows its category and trust label. Synthetic demo data.

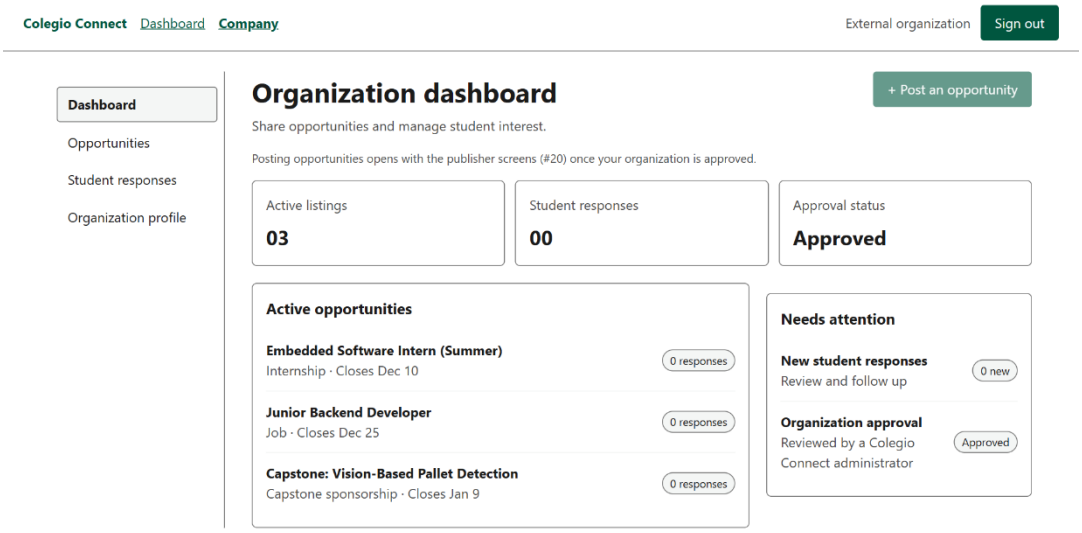


Figure 8. Company dashboard for an approved external organization, showing its active listings and approval status. Synthetic demo data.

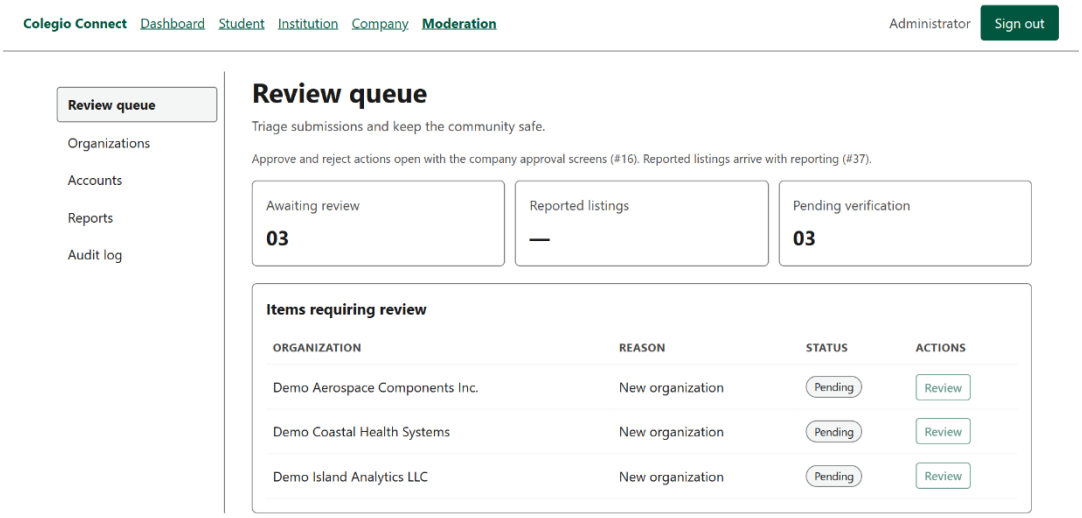


Figure 9. Moderator review queue listing companies awaiting approval. The approve and reject actions are not yet connected (Section 5.4). Synthetic demo data.

Table 4. Recorded test results.

Date	Verification / revision	Observed result	Scope
Nov 10	Hosted CI on main f0c4efe (run 42939710078)	Backend, Frontend, Container, and CI Gate passed; 112/112 backend tests; 177/177 frontend tests, lint, typecheck, build	Functional and integration tests; no load, ZAP, or accessibility scan

Date	Verification / revision	Observed result	Scope
Nov 10	Application walkthrough, main f0c4efe, isolated local Compose	Synthetic accounts for all four roles signed in; feed showed 7 listings across 6 categories with trust labels; keyword filter worked; student denied moderation area	Synthetic data inserted directly; not a deployed or usability test
Nov 10	Authentication benchmark, main e5cc181	210/210 expected HTTP 200 on sequential/five-client authentication workloads (Table 6).	Not a discovery/I3 benchmark; no sustained throughput or CPU/RAM utilization.
Nov 10	CI trigger PR #107 (head 22bbeae), not merged	Four hosted jobs passed for push and PR runs	Branch only
Nov 7	Local PostgreSQL recovery, main application e5cc181	3/3 restarts: health UP and selected count/checksum matched; fresh login mean 1.090 s.	Two synthetic identity records; not deployed recovery or full data durability.
Oct 26	Earlier CI/database integration revision 94ca852	CI passed; V1 applied on empty PostgreSQL and schema validated.	Earlier revision; superseded by the main run above

The 112 backend tests on f0c4efe include the authorization, audit, company-review, and role-management tests that earlier ran on separate branches, so this total replaces the branch counts reported previously. They also include the proposal's 18-case discovery search matrix across categories, source labels, and filters, run against test fixtures, and checks that hidden, rejected, expired, and pending-company listings are excluded. The walkthrough confirmed the trust rule in practice: listings first seeded with a moderator as publisher stayed hidden until the publisher held a role in the listing's organization. It also exposed a gap: registration does not assign the STUDENT role and no API can assign it, so a newly registered student cannot yet open the feed. The demo assigned the role directly in the database.

7.2 Results against indicators

Table 5. Indicator status. "Partial" means part of the indicator has been tested; it has not yet passed.

Indicator	Target	Evidence	Status	Next step
I1–I4	Six-category creation and discovery; 18 search cases; I3 load target; save and revisit	Six-category feed with filters on main; 18-case search matrix passes on test fixtures; no listing creation or saving	Partial	Listing creation and saving; search cases on created listings; k6 run
I5	Reject non-UPRM emails and invalid/expired codes; rate-limit verification	Registration integration tests and form-state tests (#116), both on main	Partial	Full cross-layer browser test
I6	≥ 10 negative business-authorization cases, all pass	Policy, role-management, company-review, and dashboard denials on main; publishing and consent paths absent	Partial	Run at least ten protected business-action cases after listings, responses and sponsorship exist.
I7	Zero private student fields in browse/search/save/unauthorized responses	Feed exists; response API not built	Not started	Field inventory; scan feed and response payloads
I8–I9	Every administrative action audited; report/hide/remove end to end without an office	Company decisions and role changes audited on main; reporting not implemented	Partial	Role, restriction, moderation, sponsorship actions; full workflow test

Indicator	Target	Evidence	Status	Next step
I10–I12	Two sponsorship routes; all required messages sent	Verification email and notification delivery (#115) on main	Partial (mail only)	Sponsorship routes, contact gate, full message matrix
I13	20 deployed scenarios, zero unhandled errors, CI green	CI green on main f0c4efe	Not started	Define and run deployed suite
I14–I15	Zero High/Critical ZAP and Critical/Serious axe findings; keyboard completion	Component tests, accessibility conventions	Not started	ZAP, axe, manual keyboard tasks
I16	Healthy < 5 min after restart, no data loss	Local: 1.09 s mean, 3/3 trials intact [16]	Partial (local)	Repeat on deployment with representative data
I17	≥ 6 representative users, ≥ 80% task success	—	Not started	Advisor-approved protocol, then sessions

7.3 Preliminary performance evaluation

Authentication benchmark. On November 10 the team ran the main-branch application (e5cc181) in a fresh local Docker Compose project with one verified synthetic user and no listings [16]. After warm-up (10 authenticated reads, 5 logins), a Python script timed 60 sequential and 100 five-client GET /api/auth/me requests, then 20 sequential and 30 five-client POST /api/auth/login requests. The CSRF token fetch before each login was excluded from its timing. p95 is the nearest-rank percentile; throughput is requests divided by wall time. Docker had 16 CPUs and 16.3 GB of memory allocated; CPU and memory utilization were not sampled.

Table 6. Local authentication latency and throughput, November 10, 2026.

Workload	Requests	Wall time (s)	Median (ms)	p95 (ms)	Requests/s
GET /me, 1 client	60	1.749	26.1	46.0	34.3
GET /me, 5 clients	100	1.816	81.2	122.3	55.1
POST /login, 1 client	20	1.096	53.0	64.2	18.3
POST /login, 5 clients	30	0.347	53.9	76.9	86.4

All 210 requests returned HTTP 200. With five concurrent clients, /me median latency rose from 26 to 81 ms and p95 from 46 to 122 ms. A longer run with resource sampling is needed to identify the cause. These are authentication endpoints on an empty dataset, so they say nothing about whether discovery will meet I3's two-second target. The five-client login burst lasted only 0.35 s, so its throughput is not a steady-state capacity figure.

Restart recovery. Across three trials of restarting PostgreSQL in an isolated Compose project, the database was ready after a mean of 1.015 s (0.981–1.038 s) and a fresh login succeeded after a mean of 1.090 s (1.054–1.111 s), both measured from the restart. Every trial showed a new PostgreSQL start time, a healthy status, and matching record counts and checksums [16]. This is well within I16's five-minute target, though it used only two synthetic records.

Interpretation. In a small local setup, the identity foundation responded quickly and kept its data across a database restart. Discovery latency (I3) and deployed recovery (I16) still require the full synthetic dataset and a deployed host.

8. Project Documentation and Next Steps

8.1 Repository and project web page

- **Source repository:** https://github.com/1uismar33r0/Colegio_Connect [15]. The repository is public.

- **Project web page:** https://1uismar33r0.github.io/colegio-connect-site/ (Figure 10). The page presents the project overview, milestone status, screenshots of the current build, the architecture, the interactive wireframes, the team, and links to this report and the proposal.
- **Setup and deployment:** README, contributor guide, docs/environment.md, docs/deployment.md.
- **Requirements and evidence:** approved proposal, requirements traceability register, test-evidence register, authorization policy matrix, decision record, bug log, and the performance evidence record with benchmark script and raw per-request data [10], [14], [16].

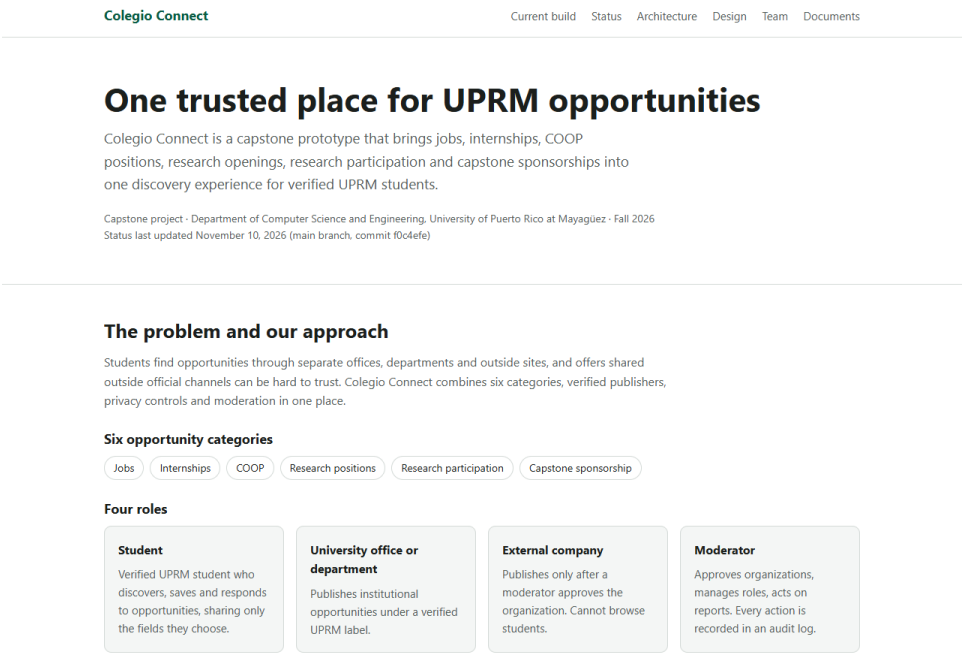


Figure 10. Project web page.

8.2 Next steps

1. **Close remaining integration items.** Assign the STUDENT role at registration, add an endpoint that lists a user's organizations, record the company applicant at registration, and decide PRs #107, #113, and #114.
2. **Finish the discovery workflow.** Listing creation with membership and approval checks, plus listing details and saving. This unblocks I1–I4, I7, and the I3 load test.
3. **Responses, moderation, and sponsorship.** Implement each with endpoint-level denial and data-minimization tests; contact release and sponsorship accept/decline get paired implementation and a separate security review.
4. **Deploy and measure.** A Render Free staging setup (one service serving the React app and API, plus managed PostgreSQL) was built and tested locally; the first public deploy was not yet working at this checkpoint. Once it is, run the deployed acceptance suite, k6, ZAP, accessibility, recovery, and usability tests against I1â€“I17.

The final presentation and elevator-pitch video are part of the final deliverables.

9. References

[1] Colegio Connect team, *Colegio Connect Capstone Proposal*, approved proposal, Univ. of Puerto Rico at Mayagüez, Fall 2026. Repository: approved-proposal.pdf.

[2] Recinto Universitario de Mayagüez, Oficina de Colocaciones, "Company Services." [Online]. Available: https://www.uprm.edu/placement/en/company-services/

[3] Recinto Universitario de Mayagüez, College of Engineering, "Open Capstone." [Online]. Available: https://www.uprm.edu/engineering/academic-affairs-2/outreach-programs/open-capstone/

[4] J. M. Nunley, A. Pugh, N. Romero, and R. A. Seals, Jr., "College major, internship experience, and employment opportunities: Estimates from a résumé audit," *Labour Economics*, vol. 38, pp. 37–46, 2016, doi: 10.1016/j.labeco.2015.11.002.

[5] S. H. Russell, M. P. Hancock, and J. McCullough, "Benefits of undergraduate research experiences," *Science*, vol. 316, no. 5824, pp. 548–549, 2007, doi: 10.1126/science.1140384.

[6] L. Fair, "Job scams targeting college students are getting personal," Federal Trade Commission, Dec. 27, 2023. [Online]. Available: https://consumer.ftc.gov/consumer-alerts/2023/12/job-scams-targeting-college-students-are-getting-personal

[7] UPRM Capstone course, "Capstone Deliverables, Fall 2026," course document, 2026.

[8] UPRM Capstone course, "Capstone Reports Guidelines," course document, 2026.

[9] F. Torre, "Evaluation of the Colegio Connect capstone proposal," UPRM Capstone course feedback, Fall 2026.

[10] Colegio Connect team, "Requirements and implementation traceability," project repository document, updated Nov. 2026.

[11] Recinto Universitario de Mayagüez, Department of Computer Science and Engineering, "PRJobs," Jun. 25, 2024. [Online]. Available: https://www.uprm.edu/cse/2024/06/25/prjobs/

[12] Recinto Universitario de Mayagüez, Department of Computer Science and Engineering, "UPRM Research Portal (RP)," Jun. 25, 2024. [Online]. Available: https://www.uprm.edu/cse/2024/06/25/uprm-research-portal-rp/

[13] M. DeLorme, "The Capstone Marketplace: An online tool for matching capstone design students to sponsors with challenging problems," in *Proc. 2015 ASEE Annu. Conf. & Expo.*, Seattle, WA, 2015, doi: 10.18260/p.24847.

[14] Colegio Connect team, "Test and verification evidence" and "CI and database integration checkpoint," project repository documents, Nov. 2026.

[15] Colegio Connect team, "Colegio_Connect source repository," main branch at commit f0c4efe, Nov. 10, 2026. [Online]. Available: https://github.com/1uismar33r0/Colegio_Connect

[16] Colegio Connect team, "Preliminary performance evidence, November 10, 2026," with benchmark script, raw request results, and Issue #44 recovery record (commit be0946e), project repository.

Appendix A. Evidence Register

Item	Branch / PR	Revision / relevant code	Hosted CI / verification	Owner
Baseline	main	f0c4efe	42939710078; four jobs passed; 112 backend, 177 frontend tests	Team
Security, audit, company review	#118, #119, #120	merges 36157e6, 2267d52, 3500ba9	Pre-merge run 42896400510 (#119 head 265fd40); on main	Gabriel
Role management, first-admin bootstrap	#124	merge f68480c	On main	Cristian
Roles in login and /me	#125	merge 18ae2ba	On main	Cristian
Authentication screens	#116	merge 6930bc4	On main	Luis
Notification delivery	#115	merge 6ac9508	On main	Luis
Listing schema, feed, dashboard APIs	#127	merge 309ffa9	On main	Gabriel
Role dashboards	#128	merge f0c4efe	On main	Luis
Recovery rehearsal	#114 / Issue #44 (open)	script c259a70; app e5cc181; record be0946e	3 local trials	Gabriel
CI trigger coverage	#107 (open)	head 22bbeae	42897093682 (push), 42897103159 (PR); four jobs each	Gabriel
Authentication benchmark	main-based test	e5cc181	210 local measured requests, Nov 10	Team
Earlier CI and DB integration	historical	94ca852	PostgreSQL V1 and CI at earlier revision	Gabriel

Local test environment: Windows, Java 25.0.5, Docker Desktop 29.5.2, PostgreSQL 18.6 via Testcontainers. Screenshots: headless Chrome driven by Playwright against an isolated Docker Compose project running f0c4efe.